Neurocomputing 416 (2020) 340–351

Contents lists available at ScienceDirect

Neurocomputing

journal homepage: www.elsevier.com/locate/neucom

Reactive Soft Prototype Computing for Concept Drift Streams

Christoph Raab*, Moritz Heusinger, Frank-Michael Schleif

University for Applied Sciences Würzburg-Schweinfurt, Sanderheinrichsleitenweg 20, Würzburg, Germany

ARTICLE INFO

Article history:
Received 15 July 2019
Revised 29 October 2019
Accepted 30 November 2019
Available online 8 April 2020

Keywords:
Stream Classification
Concept Drift
Robust Soft Learning Vector Quantization
Kolmogorov-Smirnov
RMSprop
Momentum-Based Gradient Descent
Prototype Adaptation

ABSTRACT

The amount of real-time communication between agents in an information system has increased rapidly since the beginning of the decade. This is because the use of these systems, e.g. social media, has become commonplace in today's society. This requires analytical algorithms to learn and predict this stream of information in real-time. The nature of these systems is non-static and can be explained, among other things, by the fast pace of trends. This creates an environment in which algorithms must recognize changes and adapt. Recent work shows vital research in the field, but mainly lack stable performance during model adaptation. In this work, a concept drift detection strategy followed by a prototype-based adaptation strategy is proposed. Validated through experimental results on a variety of typical non-static data, our solution provides stable and quick adjustments in times of change.

1. Introduction

Recent years demonstrated a rapidly increasing amount of data generated by technologies like social media or sensor data. In particular, data is streamed and exceeds the memory and processing capabilities of analyzing systems by far. Hence, streaming algorithms are designed to process data as fast as they arrive, online and without storing large portions of data in the main memory. In a supervised setting, streams are often affected by a change of underlying class distributions known as concept drift [35]. This results in drops of the prediction performance of prior models, making them unusable. The detection and handling of these events is one key area in the field of streaming research. Drifts appear differently through speed, intensity, and frequency, i.e. incremental, abrupt, gradual or reoccurring shown in Fig. 1.

The stability-plasticity dilemma [1] defines the trade-off between incorporating new knowledge into models (plasticity) and preserve prior knowledge (stability). This prevents stable performance over time because on the edge of a drift, significant efforts going into learning and testing against new distributions.

The main contribution of this work is a concept drift streaming algorithm able to maintain stability during drift while learning new concepts. The Robust Soft Learning Vector Quantization (RSLVQ) [2], recently considered as stream classifier [3], is enhanced by a prototype adaptation technique and combined with the Kolmogorov-Smirnov (KS)-Test for concept drift detection and referred as Reactive Robust Soft Learning Vector Quantization (RRSLVQ). The RRSLVQ is tested against standard concept drift stream classifiers on benchmark streams and showed stability during the drift, while quickly learning new concepts. Further, frequent reoccurring concept drift, which has not yet been considered in the literature, is introduced in Section 3.2 and integrated into the study.

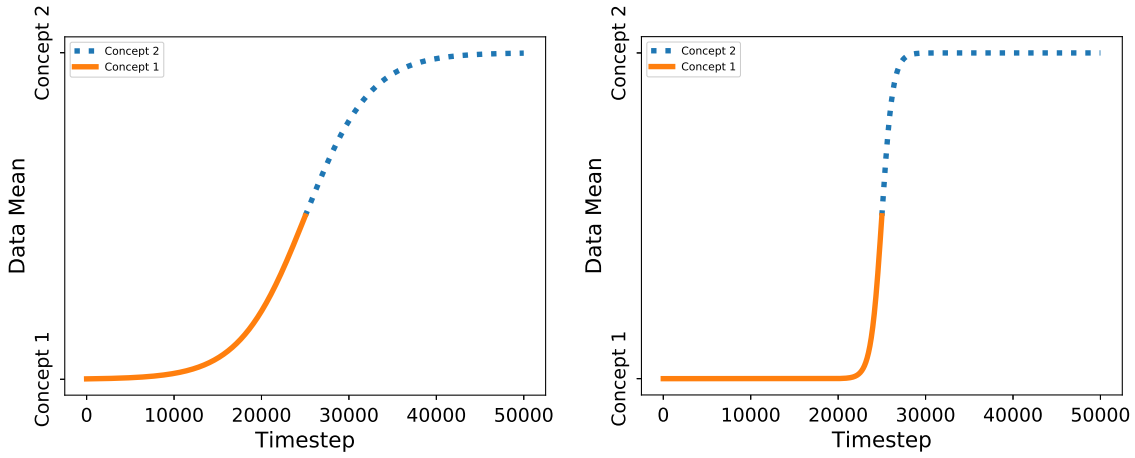
The remaining structure of the paper is given in the following. We discuss prior work on modifications of LVQ-prototypes and concept drift detectors in Section 2. In Section 3, stream classification and concept drift are introduced. The RRSLVQ as the main contribution is discussed in Section 4 and is divided into two parts. The concept drift detector is described in Section 4.1. The prototype adaptation strategy is discussed in Section 4.3 and its properties are shown in 4.4. An experimental study in Section 5 validates the approach.

2. Related Work

Concept drift detectors are trying to detect a change in streams either by monitoring the distribution of the streams or the performance of a classifier with respect to some benchmark, e.g. accuracy. A popular approach for monitoring the prediction accuracy of a classifier is the Adaptive Sliding Window (Adwin)[4] and it assumes that if a change in performance is detected, the concept has changed. Adwin identifies changes in distributions using a window W and splits W into two adaptive subwindows and compares underlying statistics. The main window grows as there

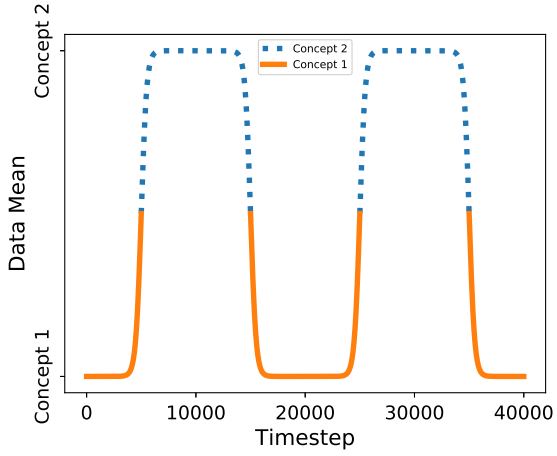
* Corresponding author.
E-mail addresses: christoph.raab@fhws.de (C. Raab), moritz.heusinger@fhws.de (M. Heusinger), frank.schleif@fhws.de (F.-M. Schleif).

https://doi.org/10.1016/j.neucom.2019.11.111
0925-2312/

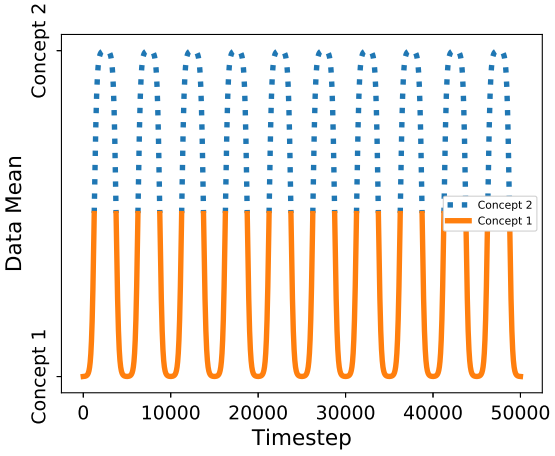


(a) Incremental Drift

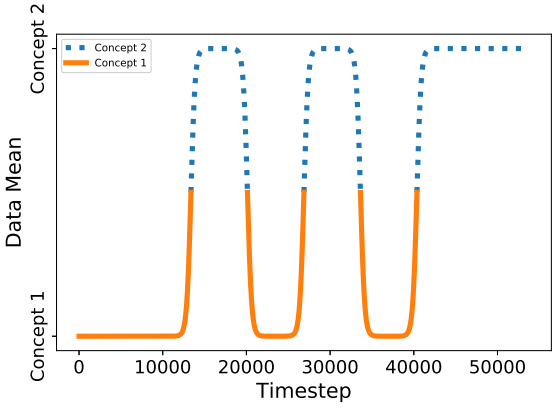
(b) Abrupt Drift



(c) Reoccurring Drift



(d) Frequent Reoccurring



(e) Gradual Drift

Fig. 1. Different types of drifts, one per sub-figure and illustrated as data mean. The colors mark the dominate concept at given time step. The vertical axis shows the data mean and the transition from one to another concept. Given the time axis the speed of the transition is given, which reaches from very slow at incremental to very fast at frequent reoccurring drift. The figures are inspired by [1].

is no change detected and shrinks if a change between statistics of subwindows is detected. The change is recognized via Hoeffding Bound [4].

Other methods similar to our approach identify drift not only but mainly via statistical tests on distributions of streams. In [5], two windows are maintained by a randomized search tree, which keeps recent data and the last concept. The KS-Test detects concept drift between windows. The approach simplifies the KS-Test similar to our approach. However, it requires a table of critical values for application. Further research is done in [6], where one window stores a snapshot of the concept since last drift and another store a recently surveyed concept. This detector is supervised and is not

able to detect concept drift independent of conditional class probabilities.

Concept drift handling techniques can be roughly divided into active and passive [7]. The passive ones have no specific detection strategy, hence continually updating the model without awareness of concept drift. Active adaptation changes a model noticeable. By means of Learning Vector Quantization (LVQ), prototype adaptation or insertion is common [8].

The family of LVQ algorithms is a learning scheme of prototypes representing class regions [9]. The prototypes have a geometric representation and provide a simple interpretation and classification scheme. Within the learning process, the prototypes are attracted and repelled depending on the class of given samples minimizing the error function. Note that the first version of LVQ is a heuristic-based approach. However, through recent developments in the field, the Generalized-LVQ (GLVQ) [10] or probabilistic approaches like RSLVQ [2] are standard [11].

In [12], a new prototype is inserted as the mean of a set of misclassified examples. Losing et al. [13] select one sample from given samples as new prototypes, which minimizes the error. A nearmean technique is proposed by Climer et al. [8]. It uses a specific sample as a new prototype, which has the smallest Euclidean distance to the mean of a set of misclassified examples. Besides great results in the respective domains, they share the problem of assuming the existence of all classes in a given batch of samples or apply asymmetric prototype insertion. Due to the composition of streams, with potentially unbalanced classes [14], this assumption is not guaranteed. In this work, we propose a prototype adaptation strategy that can cope with missing classes.

3. Preliminaries

3.1. Stream Classification

In supervised classification a stream with potentially infinite length is a sequence $S = \{s_1, \dots, s_t, \dots\}$ of tuples $s_i = \{\mathbf{x_i}, y_i\}$, arriving one s_t at time t. A stream classifier predicts labels $y_t \in \{1, \dots, C\}$ of unseen data $\mathbf{x}_t \in \mathbb{R}^d$ by prior model $\hat{y} = h_{t-1}(\mathbf{x}_t)$ and includes this tuple into the model afterwards $h_t = learn(h_{t-1}, s_t)$. This requires algorithms to predict and learn at any time.

Further, a stream classifier must cope with non-stationary environments, which change through the occurrence of concept drift.

3.2. Concept Drift

Concept drift is the change of joint distributions of a set of samples and corresponding labels between two points in time by

$$\exists \mathbf{X} : p_{t-1}(\mathbf{X}, y) \neq p_t(\mathbf{X}, y). \tag{1}$$

There are a variety of different drift types occurring in streaming data and causes the concept to change from one time step to another one. Fig. 1 gives a short overview of drift types [1]. Every sub-fgure shows a particular drift given as data mean. The shapes mark the dominating concept at a given time step. The vertical axis shows the overall data mean and the transition from one to another concept. The goal of detectors is to identify any change as soon as possible, as in Eq. (1). Therefore a detector should monitor the data distribution rather than performance values.

Note that we can rewrite Eq. (1) to

$$p_{t-1}(y|\mathbf{X})\, p_{t-1}(\mathbf{X}) \neq p_t(y|\mathbf{X})\, p_t(\mathbf{X}). \tag{2}$$

If only the prior distribution $p(\mathbf{X})$ changes for two points in time, then it is called virtual drift. We assume that a change in this prior distribution is always present at real concept drift [1]. Therefore, we assume with the proposed detector to identify every concept drift through monitoring $p(\mathbf{X})$. This is also an essential assumption for most statistical tests since they do not observe the conditional class probability but the prior distribution.

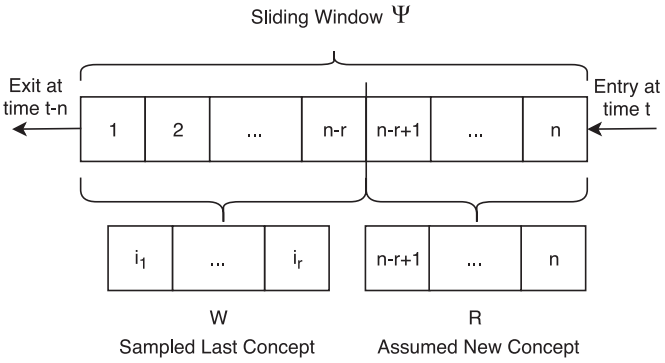


Fig. 2. The memory strategy within the concept drift detector. It stores n samples in a sliding window. At every time step, r samples are picked uniform and are compared against the newest r samples of the window.

The frequent reoccurring drift is shown in the Fig. 1d and has not yet been considered in the literature but is particularly impressive. Stream classification algorithms are usually tested on streams with a low number of drifts per setting [15]. However, in real-world settings like robotics or autonomous driving, the concept can change very often due to changing external conditions like lighting or weather [16,17]. Therefore, we also use stream generators with frequent reoccurring drifts.

4. Reactive Robust Soft Learning Vector Quantization

The section details the main contribution and contains two main parts. The first in Section 4.1 details the concept drift detector and the second part introduces the RSLVQ (Section 4.2), the proposed prototype adaptation strategy (Section 4.3) and the properties of the adaptation (Section 4.4).

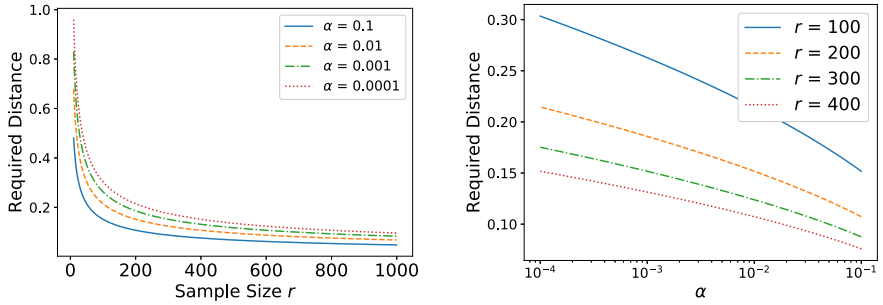
4.1. Concept Drift Detection

Before applying a drift detector, a memory strategy must be defined. The sliding window Ψ keeps n recent points from the stream. It pushes incoming data to the top and removing the oldest one from the bottom. We use the Kolmogorov-Smirnov test as a concept drift detector, which expects two data windows. Therefore, we create two windows out of the sliding window Ψ. The first windows $R = \{\mathbf{x}_i \in \Psi\}_{i=n-r+1}^{n}$ has the most recent data points from Ψ. We define most recent as the r newest arrived data points. The second window W is created by sampling uniformly from the non-recent part of Ψ by

$$W = \{\mathbf{x}_i \in \Psi | i < n - r + 1 \wedge p(\mathbf{x}) = \mathcal{U}(\mathbf{x}_i | 1, n - r)\} \tag{3}$$

with $|W| = |R| = r$ and $\mathcal{U}(\mathbf{x}_i|1, n-r) = \frac{1}{n-r}$ is the uniform distribution. By this, we do not make any assumptions of distribution but assume to represent the concept of the non-recent data. The memory strategy with the sliding window is summarized in Fig. 2. For subsequent adaptation steps, we also store the label y_i to a given sample $\mathbf{x}_i$.

By using the above scheme, there is a limitation in concept drift detection regarding some streams. The worst case is to take a sample from a recurring stream at certain intervals and always get the same concept of the changing stream. Thus the distribution seems to be the same from window R and the samples W, because the detector always picks at the same rate of concept change. However, there is clearly a concept drift ongoing. This is due to the large window and the Uniform picking. Other probability distributions for the sampling scheme should lead to a different result, but



(a) Window Size (Log scale) (b) Confidence Level (Log Scale)

Fig. 3. Plot of the parameter sensitivity to the required distance of the KS-Test in log scale on the x-axes. The left figure shows the sample size and the right figure, shows the confidence level against the required distance needed for a concept drift detection. Both figures show sensitivity towards the window size and especially with a tiny window the test becomes more insensitive to small differences in distribution.

with familiar problems and some concept drifts will be missed, because the sampling procedure is an approximation of the window Ψ. We choose the Uniform distribution because it has no parameters and is fair in sampling from unknown distributions.

The Kolmogorov-Smirnov [18] test is a non-parametric test accepting one-dimensional data without any assumptions of underlying distributions. It compares the absolute distance $dist_{w,r}$ between two empirical cumulative distributions F_W and F_R by

$$dist_{w,r} = \sup_x |F_W(x) - F_R(x)|, \tag{4}$$

where $F_{(\cdot)}(x) = \frac{1}{n}\sum_{j=1}^{n} I_{[-\infty,x]}(X_j)$ and $I_{[-\infty,x]}(X_j)$ is an indicator function which is one if $X_i \geq x$ and zero otherwise. Note that $sup(x)$ is the smallest required x for a condition to be valid. If this lower bound of maximum distance, i.e. $dist_{w,r}$, is greater than the test statistic, then the null hypothesis is rejected, with significance level α. For two subwindows W, R with the same size, the test is reduced to

$$dist_{w,r} > c(\alpha)\sqrt{\frac{n+r}{nr}} = \sqrt{-\frac{1}{2}\ln\alpha}\sqrt{\frac{n+r}{nr}} \overset{(n=r)}{=} \sqrt{-\frac{\ln\alpha}{r}}, \tag{5}$$

where α is the confidence level, r is the size of the window R and n is the size of the window w. $c(\alpha)$ is the critical value of the test with respect to the confidence level α, which can either be looked up for standard test sizes or computed by $\sqrt{-\frac{1}{2}\ln\alpha}$. Due to the restriction to one-dimensional distributions, the test in Eq. (5) is applied to all dimensions, therefore at any point t, d tests must be done due to $\mathbb{R}^d$. With this extension and based on Eq. (5), the following can be stated:

Lemma 4.1. *Given* $\mathbf{X}_t$ *and* $\mathbf{X}_{t-1}$ *with* $\mathbf{X}_k = \{\mathbf{x}_j\}_{j=1}^{n} \in \mathbb{R}^d$, $p(\mathbf{X}) = \prod_{i=1}^{d} p(\mathbf{x}^{(i)})$, $\forall \mathbf{x}^{(i)}$ *~ i.i.d and their cumulative distributions* F_t, F_{t-1}, *KS-Test detects any change between* $p(\mathbf{X}_t)$ *and* $p(\mathbf{X}_{t-1})$, *i.e.* $\exists \mathbf{x}^{(i)}$: $p_t(\mathbf{x}^{(i)}) \neq p_{t-1}(\mathbf{x}^{(i)})$, *with probability* $1-\alpha$, *if the difference in empirical distribution is at least* $\sqrt{\frac{-ln\alpha}{r}}$.

To implement Lemma 4.1, we set $\mathbf{X}_t = R$ and $\mathbf{X}_{t-1} = W$. For a reasonable choice of window size r, the influence of changing the window size and the confidence level of the KS-Test is demonstrated in Fig. 3. The left figure shows the influence of the window size and the right figure, the confidence level on the required distance. By decreasing the confidence level α, *dist* increases, and with increasing window size r, *dist* decreases. Hence, they behave competitively w.r.t. *dist* value. In a streaming context with potentially unlimited data, the KS-Test will be too sensitive, given a large window size, with many false positives. This motivates the choice of a relatively small window size $r = 30$, but still being statistically valid at the same time.

However, when it comes to many statistical tests, the problem with multiple hypothesis testing arises, with the consequence of false positives due to random chance. By applying the Bonferroni-Dunn correction [19], we reduce this effect by obtaining a new alpha with $\alpha^* = \lfloor \frac{\alpha}{r} \rfloor = \frac{0.05}{60} = 0.0001$. Because α^* affects Eq. (5), a small alpha increases the required distance.

Both, a small window size and the corrected alpha should avoid false positives. However, both actions do not entirely avoid false signals, but making the KS-Test more insensitive and combined with the proposed prototype adaptation strategy false positives are not critical. In the following, the KS-Test detector is called Kswin.

4.2. Robust Soft Learning Vector Quantization

The Robust Soft Learning Vector Quantization [2] is a probabilistic prototype based classification algorithm and capable of online learning. Given a labeled dataset $\mathbf{D} = \{(\mathbf{x_i}, y_i) \in \mathbb{R}^d \times \{1,\ldots,C\}\}_{i=1}^{n}$ as classification task. The RSLVQ assumes that $\mathbf{D}$ can be represented as class-dependent Gaussian mixture model and approximates this mixture by a set of m prototypes $\Theta = \{(\boldsymbol{\theta}_j, y_j) \in \mathbb{R}^d \times \{1,\ldots,C\}\}_{j=1}^{m}$, where each prototype represents a multi-variate Gaussian model, i.e. $\mathcal{N}(\boldsymbol{\theta}_j, \Sigma)$ with $\Sigma = \mathbf{I}\sigma$ and $\mathbf{I}$ as identity matrix. Further, $\boldsymbol{\theta}_j$ is a d-dimensional prototype representing the mean of a Gaussian mixture component with a variance of σ, which is assumed to be the same for every component. The goal of RSLVQ algorithms is to learn prototypes representing the class-dependent distribution, i.e. a corresponding class sample $\mathbf{x}_i$ should be mapped to the correct class or Gaussian mixture based on the highest probability.

The RSLVQ [2] algorithm minimizes the objective function

$$\mathcal{L} = \frac{1}{n}\sum_{i=1}^{n} ls(\mathbf{x}_i, y_i|\Theta) \text{ with } ls(\mathbf{x}_i, y_i|\Theta) = \frac{p(\mathbf{x}_i, \bar{y}_i|\Theta)}{p(\mathbf{x}_i|\Theta)}. \tag{6}$$

Where $p(\mathbf{x}_i, \bar{y}_i|\Theta)$ is the probability density function that $\mathbf{x}_i$ is generated by the mixture model of any of the different classes and $p(\mathbf{x}_i|\Theta)$ is the overall probability density function of $\mathbf{x}_i$ given Θ. In other words, $\bar{y}_i$ is every label which is not the ground truth label y_i of x_i. At time step t, these probabilities are computed by

$$ls_t(\mathbf{x}_t, y_t|\Theta) = \frac{p(\mathbf{x}_t, \bar{y}_t|\Theta)}{p(\mathbf{x}_t|\Theta)} = \sum_{j:c_j \neq y_t} P(j|\mathbf{x}_t). \tag{7}$$

Where j is the j-th prototype in Θ and c_j is the corresponding label. $P(j|\mathbf{x})$ is the probability that $\mathbf{x}$ is generated by the component j with

$$P(j|\mathbf{x}) = \frac{p(\mathbf{x}|j)P(j)}{p(\mathbf{x})} = \frac{exp\left(-\frac{\left\|\mathbf{x}-\boldsymbol{\theta}_j^2\right\|}{2\sigma^2}\right)}{\sum_{k:c_k \neq c_j} exp\left(-\frac{\left\|\mathbf{x}-\boldsymbol{\theta}_k^2\right\|}{2\sigma^2}\right)}. \tag{8}$$

Algorithm 1 Reactive Robust Soft Learning Vector Quantization (RRSLVQ).

State 1– Initialization

Input: m as number of prototypes; σ as width of Gaussian kernel; γ as decay-factor; r as sampling size; α as confidence-level.
Output: $h_0(\mathbf{x}; \Theta, \gamma, \alpha, r)$ as initialized model
1: Create prototypes $\Theta = \{\boldsymbol{\theta}_j\}_{j=1}^{m} \sim \mathcal{N}(\mathbf{0}, \Sigma = \mathbf{I}\sigma)$
State 2– Prediction

Input: $h_{t-1}(\mathbf{x})$ as RRSLVQ model; $\mathbf{x}_t \in \mathbb{R}^d$ sample point arrived at time t.
Output: Predicted label $\hat{y}_t$ of sample $\mathbf{x}_t$.
2: Compute nearest prototype $\boldsymbol{\theta}_q$ to $\mathbf{x}_t$ (Eq.(13))
3: Assign predicted label $\hat{y}$ based on the neared prototype $\boldsymbol{\theta}_q$
State 3– Learning

Input: $h_{t-1}(\mathbf{x})$ as RRSLVQ model from previous time step; $\{\mathbf{x}_t, y_t\}$ as labeled data at time step t.
Output: $h_t(\mathbf{x})$ as updated model.
4: Update sliding window (According to Fig. 2)
5: Create sampling window W and R(Eq. 3 and Fig. 2)
6: Compute d distribution differences $dist_{w,r}$ between $R, W \in R^d$ (Eq.(4))
7: **if** $\exists d_i > \sqrt{-\frac{\ln \alpha}{r}}$(CD-Detection Eq. (5)) **then**
8: **for** $\forall \mathbf{x}_i, y_i \in R$ **do**
9: Step 12 to 16
10: **end for**
11: **end if**
12: **for** $\boldsymbol{\theta}_j : j = 1 \rightarrow m$ **do**
13: Compute posterior probabilities (Eq. (8))
14: Compute gradient g_t for $\boldsymbol{\theta}_j$ (Eq. (9))
15: Compute past gradient and parameter updates (Eq. 11 and 12)
16: Update prototype $\boldsymbol{\theta}_{j+1} = \boldsymbol{\theta}_j - \Delta\boldsymbol{\theta}_t$ (Eq. 10)
17: **end for**

Based on this, the gradient [2] for the prototype update step at time t is computed by

$$g_t = \begin{cases} -P(j|\mathbf{x}_t) ls_t (\mathbf{x}_t - \boldsymbol{\theta}_t), & if\ c_j = y_t, \\ P(j|\mathbf{x}_t)(1 - ls_t)(\mathbf{x}_t - \boldsymbol{\theta}_t), & if\ c_j \neq y_t. \end{cases} \tag{9}$$

Note that ls_t abbreviated for Eq. (7). The normalization term in Eq. (6) is not computed at the update step and, therefore, the RSLVQ is feasible for potentially infinite streams.

The prototypes in each update step will be optimized with a momentum-based gradient technique designed for RSLVQ [20] given with

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \Delta\boldsymbol{\theta}_t = \boldsymbol{\theta}_t - \frac{\sqrt{E[\Delta\theta^2]_t + \epsilon}}{\sqrt{E[g^2]_t + \epsilon}} g_t. \tag{10}$$

Where ϵ is a small positive value for numerical stability. The $E[g^2]_t$ are the stored past squared gradients as running mean

$$E[g^2]_t = \gamma E[g^2]_{t-1} + (1 - \gamma) g_t^2 \tag{11}$$

and $E[\Delta\theta^2]$ are the past squared parameter updates as running mean

$$E[\Delta\theta^2]_t = \gamma E[\Delta\theta^2]_{t-1} + (1 - \gamma)\Delta\theta_t^2. \tag{12}$$

Both equations are controlled by the decay-factor γ controlling the relevance of previous updates and the current ones. As pointed out in [21], a momentum-based gradient descent is a reliable strategy to handle concept drift with LVQ classifiers. In a streaming setting at every time step, every prototype will be updated by Eq. (10) with a given tuple $s_i = \{\mathbf{x}_i, y_i\}$, as shown in pseudocode 1 . The RSLVQ predicts a given sample point $\mathbf{x}$ by selecting the nearest prototype $\boldsymbol{\theta}_q$ according to the Gaussian kernel

$$q = \arg\min_i exp\left(-\frac{\left\| \mathbf{x} - \boldsymbol{\theta}^2 \right\|}{2\sigma^2} \right), \tag{13}$$

and assigning the corresponding label of $\boldsymbol{\theta}_q$ to $\mathbf{x}$. The σ is the width of the Gaussian kernel and together with the number of prototypes, are the only tuneable parameters in the RSLVQ. For a more comprehensive derivation of RSLVQ with momentum SGD see [2,20].

4.3. Prototype Adaptation Strategy

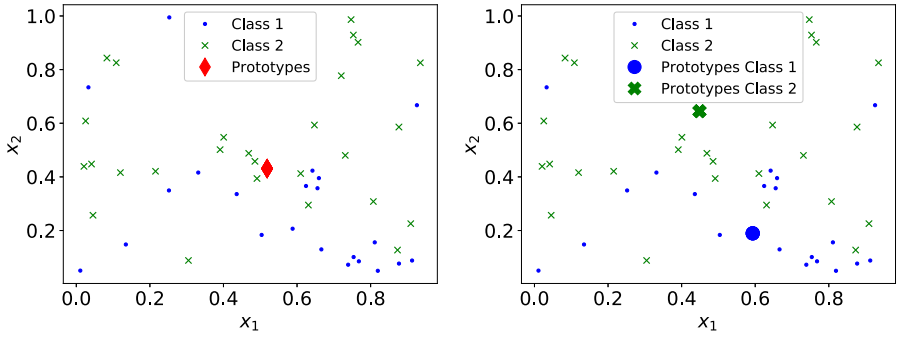
In this work, the adaptation strategy adapts actively to abrupt or gradual concept drift. We allow missing classes in R since streams have unbalanced classes and class-wise adaptation is usually infeasible. If a drift is detected as in Section 4.1, window R represents a new context and Θ represents the last context. According to Kswin, the current set of prototypes Θ has approximated an out-dated distribution.

Hence, we propose the following two-step adaptation strategy. The first step is to replace the complete set Θ of prototypes by m new prototypes. Note that the classes remain the same. A good starting point for replacement by means of approximating an unknown mixture model by Gaussian mixture is the mean of points[2], regardless of class affiliation. Therefore, the new prototypes become

$$\boldsymbol{\theta}_i^{(new)} = \frac{1}{r} \sum_{\mathbf{x} \in R} \mathbf{x}, \ \forall i = 1, \ldots, m. \tag{14}$$

The next step is to train all prototypes as in Eq. 10 on all $\{\mathbf{x}_i, y_i\} \in R$. This modifies prototypes and draws them to the same class points and pushes different class prototypes away. In the case of missing classes, the algorithm pushes the prototypes away from points with a different class.

The process is shown in pseudocode 1 in the *State 3 - Learning Rate*. Further, we visualized the adaptation in Fig. 4. It shows the adaptation of two prototypes with different classes as mean in 4a and the optimization afterward in 4b in a two-dimensional feature



(a) Insertion Step (b) Optimization Step

Fig. 4. Process of prototype adaptation strategy on given window R with $|R| = 50$ for more details. First, insertion of m prototypes as mean over all samples in the left figure, marked as a red diamond. Second after optimization, where green/blue crosses/dots are representing class prototypes.

space. Data is generated with MIXED generator stream from Scikit-Multiflow [22]. Further, we show in Section 4.4 that adaptation on R leads to a lower error, if the distributions are different enough, rather then taking no action. Therefore, false positives as described in Section 4.1, are not critical because every replacement always leads to lower error and does no harm.

4.4. Stability during Concept Drift

The stability during drift will be shown in the following and we start with the introduction of notation and background. Given a two-class classifier $h : \mathcal{X} \to \{-1, 1\}$ on a given sample $\mathbf{x} \in \mathcal{X} \subseteq \mathbb{R}^d$. The loss of a classifier $l(h(x)) \in \mathbb{R}^+$ is measuring the performance of a given set of labeled data sampled from the concept $C = p(x, y) = p(y|x)p(x)$. The expected risk $R(h(x))$ and the empirical risk $\hat{R}(h(x))$ of the classifier $h(x)$ [23] are defined as

$$R(h(x)) = \mathbb{E}[l(h(x), y)] = \int l(h(x), y)p(y|x)p(x)dxdy \leq \hat{R}(x) = \frac{1}{n}\sum_{i=1}^{n} l(h(x), y), \tag{15}$$

while computing the empirical risk with n samples.

Lemma 4.2. *Given two classifiers $h_1(x)$ and $h_2(x)$ with loss $l(h_{(\,\cdot\,)}(x))$ and trained on two different concepts C_1 and C_2 with c classes each and corresponding risks $R_{(\,\cdot\,)}(h_{(\,\cdot\,)})$. The empirical risk $R_2(h_1(x))$ on C_2 can be bounded by*

$$\hat{R}_2(h_2(x)) \leq 1 - \frac{1}{c} \leq \delta R_2(h_1(x)) \leq \hat{R}_2(h_1(x)), \tag{16}$$

under the assumption that there is a linear relationship $\delta = \frac{p_2(x)}{p_1(x)}$ with $\delta \in \mathbb{R}^+$ between the prior of the concepts.

Proof. First consider the first classifier on the concept one, where it is trained on with the risk

$$R_1(h_1(x)) = \mathbb{E}[l(h_1(x), y)] = \int l(h_1(x), y)p_1(y|x)p_1(x)dxdy, \tag{17}$$

where R_1 is the risk corresponding to concept C_1. Applying $h_1(x)$ to concept two leads to the expected risk

$$R_2(h_1(x)) = \int l(h_1(x), y)p_2(y|x)p_2(x)dxdy. \tag{18}$$

Similar as in [23], we assume the relationship between the two concepts by $p_2(x) = \delta p_1(x)$, e.g. two variants of Gaussian distributions, then we can rewrite the risk to

$$R_2(h_1(x)) = \int l(h_1(x), y)p_2(y|x)\delta p_1(x)dxdy = \delta \int l(h_1(x), y)p_2(y|x)p_1(x)dxdy. \tag{19}$$

The expected risk $R_2(h_2(x))$ is analog to Eq. (18). If the difference in prior distribution between the two concepts is large enough, then δ becomes large. Hence, for large δ, the expected risk of $R_2(h_1(x))$ becomes larger than random guessing, which is $1 - \frac{1}{c}$ for c classes. Because $h_2(x)$ is trained on concept two, we expect that the empirical risk $\hat{R}_2(h_2(x))$ in the worst-case is random and therefore

$$\hat{R}_2(h_2(x)) \leq 1 - \frac{1}{c} \leq \hat{R}_2(h_1(x)). \tag{20}$$

□

This can be implemented as $h_1(x)$ being the RSLVQ model at time $t-1$ and $h_2(x) = g(R)$ at time t after the adaptation procedure. Hence, for large enough differences between windows, which the Kswin detects given a small r and α, the performance of the adaptation yields a smaller loss in comparison to the model from $t-1$. This also means that the classification result is not negatively affected even if Kswin detects a change in prior distribution within a concept, which is not real concept drift. Therefore, the false positives of Kswin do not harm. The effect is demonstrated in Fig. 5, showing that δ is always sufficiently large so that $g(R)$ achieves better performance on the Mixed concept drift stream.

4.5. Time and Memory Complexity

The RRSLVQ optimized via online momentum gradient descent has the time complexity of $\mathcal{O}(m)$ at given time t for m prototypes without concept drift. The concept drift handling has the complexity $\mathcal{O}(r \cdot m)$. The KS-Test can be implemented in log-linear time [5]. The demand for memory depends on the number of prototypes m and sliding window size n. Therefore, the model needs at maximum $m \times d$ and $n \times d$ as real float numbers in memory, which accumulates in the complexity of $\mathcal{O}(d \cdot (m+n)) = \mathcal{O}(d \cdot (m+300))$ with $n = 300$. For using RRSLVQ in embedded systems with restrictive memory, we follow the *any memory framework* by [1, p. 6] and we suggest setting the number of prototypes plus the window length to a maximum of $d \cdot (m+300) < k$ with k as maximum float memory storage capacity.

5. Experiments

The experimental section of the paper consists of three parts. First, we provide the study design, the performance metrics and a data set overview with their properties. The second part analyses the performance of Kswin as an independent approach compared to other concept drift detectors. The last part shows the performance of the RSLVQ paired with the Kswin detector.

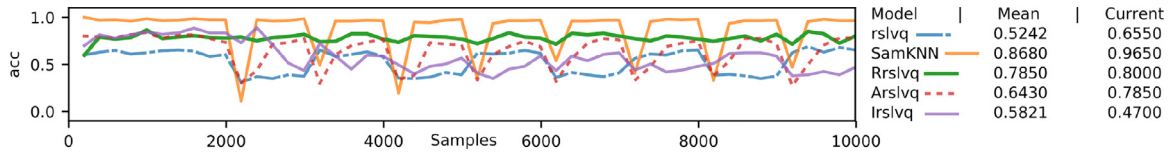


Fig. 5. Performance of baseline RSLVQ, RRSLVQ, Incremental-RSLVQ, Adwin-RSLVQ, and SamKNN [7] on Mixed stream in SciKit-Multiflow [22]. Plot shows clear drops in performance of non-RRSLVQ methods during abrupt concept drift. From point 2000, drift happens every 1000 points after last drift is finished. The line shows accuracy over the last 200 samples. Best viewed in color.

Table 1
Configuration of the data sets (A: Abrupt Drift, G: Gradual Drift, I_m: Moderate Incremental Drift, I_f: Fast Incremental Drift and N: No Drift)

Data set	# Instances	# Features	Type	Drift	# Classes
SEA_a	1,000,000	3	Synthetic	A	2
SEA_g	1,000,000	3	Synthetic	G	2
$MIXED_a$	1,000,000	4	Synthetic	A	2
$MIXED_g$	1,000,000	4	Synthetic	G	2
RTG	1,000,000	10	Synthetic	N	2
RBF_m	1,000,000	10	Synthetic	I_m	5
RBF_f	1,000,000	10	Synthetic	I_f	5
HYPER	1,000,000	10	Synthetic	I_f	2
POKER	829,201	11	Real	-	10
GMSC	120,269	11	Real	-	2
ELEC	45,312	8	Real	-	2
COV	581,012	27	Real	-	7
AIR	539,383	8	Real	-	2
SQRE	200,000	2	Real	-	4

5.1. Study Design

We use six stream generators (synthetic data) and six real-world datasets. We follow the study design of [14], but additional, we simulate frequent gradual/abrupt reoccurring drift with streams having abrupt or gradual drift. This type of drift stream introduces reoccurring drift with a certain frequency. The frequency is specified in section 5.2 and section 5.3 respectively. An overview of the streams is given in Table 1. The synthetic and the real-world datasets are described in detail in Appendix A.

In a stream setting, prediction performance is measured with interleaved test-then-train accuracy [7,14]. It is the moving average accuracy including the current sequence of data at time t by

$$A(S) = \frac{1}{t}\sum_{i=1}^{t} 1(h_{i-1}(\mathbf{x}_i) = y_i). \tag{21}$$

Where $1(\cdot)$ is an indicator function and is one for a correct classification and zero otherwise. For the evaluation, the length of the streams is fixed to $t_{max} = 1,000,000$. At t_{max}, the interleaved test-then-train accuracy becomes the overall mean accuracy. We use this and the kappa-statistics for the subsequent performance tables. A stream classifier within this setting first predicts the given sequence and then learns with it. This evaluates the ability to predict and learn anytime over a long period [7].

5.2. Comparison of Concept Drift Detectors

The Kswin detector is tested against other commonly used detectors, i.e Adwin [4], Drift-Detection Method (DDM) [24] and Early Drift Detection Method (EDDM) [25]. The parameters for Kswin are $(r = 30, n = 300)$ and $\alpha = 0.0001$. The Adwin parameter is $\alpha = 0.002$. The DDM algorithm has a minimum number of test size, which is set to 30 and a detection threshold set to 3. The window size of EDDM is also set to 30 with a detection threshold of $\beta = 0.95$.

Each tested stream has 100,000 time steps with a batch size of ten per time step and in summary, there are one million samples per stream. All time steps of occurring drifts are compared against the predicted time steps of the detectors, which are either zero for no concept drift and one if drift is detected. Summarizing, there are eight standard concept drift streams with one concept drift per stream and four frequent reoccurring concept drift streams with 99,999 occurrences of concept drift per stream. The results of the concept drift detectors are split up into two parts. For both parts, the detectors test every data dimension separately and not the performance values of classifiers.

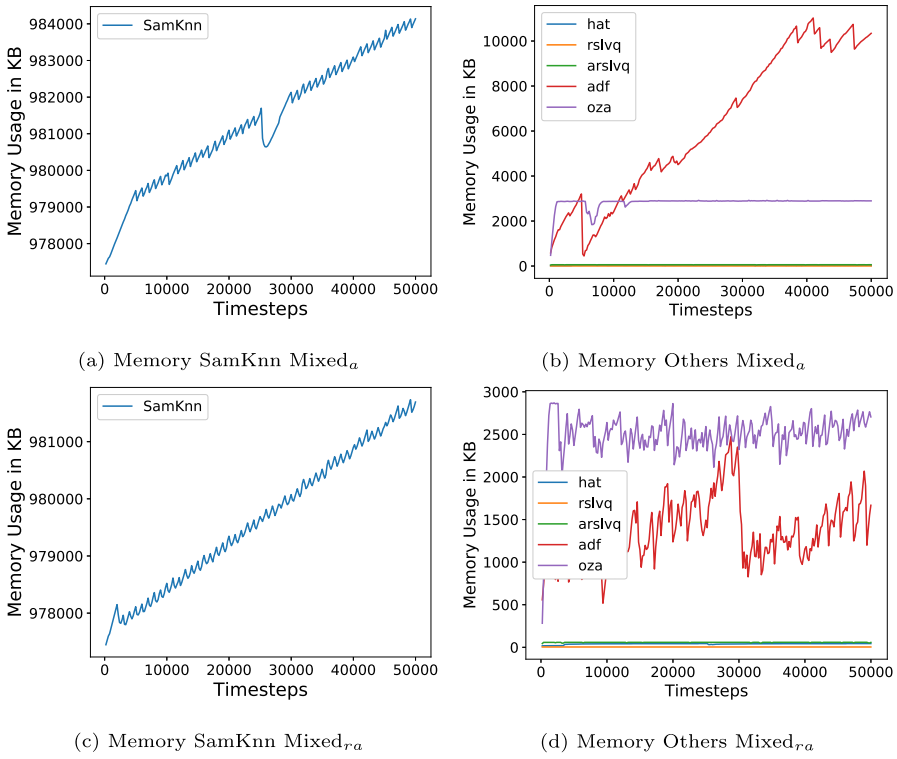
Table 2
Confusion matrix of concept drift detectors showing that Kswin detects the most drifts but also has most false positives. It is tested on standard concept streams (8) and frequent reoccurring versions (4) of them. Overall, there are 400,004 concept drifts within 12,000,000 time steps. Each of the eight standard streams has one drift, and each of the frequent reoccurring concept drift streams has 99,999 drifts.

Detectors	True Negative	False Positive	False Negative	True Positive
Kswin	748956	51040	379345	20659
Adwin	799823	173	399933	71
EDDM	799941	55	399971	33
DDM	799984	12	400004	0

Table 3
Interleaved mean accuracy of concept drift detectors with Naive Bayes as the underlying classifier. Tested on standard concept drift streams and the frequent reoccurring versions of them separated in two parts. The results are overall similar comparing prediction performance, but in the mean, the Kswin detector outperforms the remaining detectors. Winner marked bold. a = abrupt, if = incremental fast, im = incremental medium, g = gradual, ra = frequent reoccurring

Standard Streams	Kswin	Adwin	EDDM	DDM
$MIXED_a$	0.8646	**0.9024**	0.5057	0.4998
$MIXED_g$	**0.8645**	0.5004	0.5000	0.5001
Hyperplane	0.5797	**0.6090**	0.5890	0.5883
RTG	0.6325	**0.7048**	0.6318	0.6655
RBF_f	**0.6584**	0.5299	0.4952	0.5009
RBF_m	0.6459	**0.6512**	0.5034	0.6019
SEA_a	0.8400	**0.8909**	0.7036	0.8876
SEA_g	0.8334	0.8304	0.6787	**0.8396**
Standard Mean	**0.7456**	0.7089	0.5853	0.6435
$MIXED_{ra}$	**0.783**	0.4902	0.490	0.4896
$MIXED_{rg}$	**0.7766**	0.4921	0.4901	0.4975
SEA_{ra}	0.8385	**0.8766**	0.8104	0.8745
SEA_{rg}	0.8302	**0.8688**	0.806	0.847
Reoccurring Mean	**0.8217**	0.7150	0.6793	0.7065
Overall Mean	**0.7837**	0.7120	0.6323	0.6750

The results of the first part of the evaluation are given in Table 2. It shows a confusion matrix of the detectors tested on the listed concept drift stream generators given in Table 1 plus the frequent reoccurring versions. The Kswin algorithm detect the concept drifts and has, by far, the most true positives but also the most false positive. The detection accuracy of Kswin is roughly 5%, while the detection rate of remaining detectors is roughly 0.001%. Summarizing, concept drift detection per dimension on stream data is somewhat unreliable and most of the time, the detectors just missing concept drift, because of insensitivity. The Kswin detector is an improvement to the state of the art detectors by means of detection rate. The false positive rate should be tackled in



(a) Memory SamKnn Mixed$_a$ (b) Memory Others Mixed$_a$

(c) Memory SamKnn Mixed$_{ra}$ (d) Memory Others Mixed$_{ra}$

Fig. 6. Memory usage of stream classifiers tested on Mixed stream with abrupt drift (top) and frequent abrupt reoccurring drift (bottom). SAMKNN is plotted separately due to scaling issues. It shows the constant memory consumption of RSLVQ methods and HAT even during drift.

future work, but as shown in the prototype adaptation strategy in Section 4.4 and shown in the following experiments, where Kswin is paired with the Naive Bayes classifier, false positives are not critical.

The second part is shown in Table 3 and gives the prediction performance of the Naive Bayes classifier combined with a given detector. If a detector notices concept drift, the current underlying learning model of a respected detector is discarded and a new one is learned with the current batch of data. This experiment ensures that every detector uses the same underlying classifier and the classifier does not influence the performance.

Inspecting each drift separately, the Kswin algorithms is only best at four out of 16 streams. However, Kswin has the best mean performance. The second best algorithm is Adwin. The results at MIXED are interesting because the algorithms that do not recognize the concept drift, are not able to switch to the new concept. EDDM and DDM are particularly affected by this and also Adwin at frequent reoccurring MIXED. However, apart from MIXED, the performance of the detectors on the frequently reoccurring drift streams is not much worse compared to the standard streams.

5.3. Comparison of Stream Classifiers

In these experiments, we compared the RRSLVQ with Hoeffding Adaptive Tree [26], OzaBaggingAdwin [27], Adaptive Random Forest [15], SamKNN [28] and RSLVQ [2] as the baseline.

In this setting, the frequent reoccurring drifts starting at sample 2000 and, further, every 1000 samples after the last drift is finished. The width of gradual drift is 1000. For every non-frequent reoccurring stream, we are adding abrupt/gradual concept drift at position 500,000, while gradual drift has a width of 50,000.

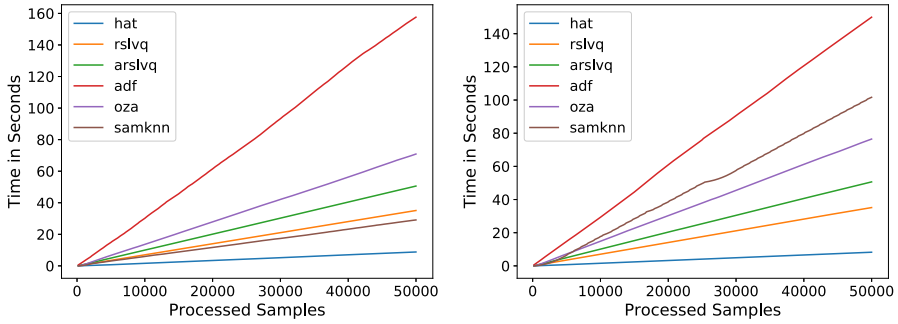
The RRSLVQ provides stable performance during drift and high adaptation rate shown in Fig. 5, superior w.r.t. other methods. Other variants of concept drift handling, like prototype insertion or prototype replacement based on the Adwin detector are not providing stability during a drift. Hence, both parts, the Kswin and the prototype adaptation strategy, are equally relevant for the stability of RRSLVQ.

An overview of memory consumption during stream processing is shown in Fig. 6. It shows that the RSLVQ variants and HAT have very low memory requirements and are stable in memory consumption during the drift. This validates Section 4.5. The stable consumption is not given by the remaining classifiers and at the detection of drifts, the memory usage fluctuates.

The time result is plotted in Fig. 7 as incremental time in seconds per processed samples. It also validates the time complexity of RRSLVQ and further shows that every other stream classifier also has linear time complexity. However, the RRSLVQ needs less total time as the ARF and OZA. The RSLVQ is slightly faster as the RRSLVQ because of missing concept drift detector, non-momentum-based SGD and prototype adaptation strategy. The HAT algorithm is the fastest. All in all, the classifiers do not need any additional time to handle the concept drift.

The results of the prediction performance of the concept drift classifiers on the data streams are presented in Table 4. Our approach shows a boost in performance to baseline RSLVQ and is comparable to other concept drift classifiers like HAT and OZA.

The RSLVQ is the worst classifier on the tested real-world datasets, affected by the worse performance on COV and SQR. The RRSLVQ performs about 8 % better when looking at the mean of the real-world datasets. However, it is still worse than the other classifiers, especially SAMKNN, which performed best. Especially on COV, where RSLVQ had an accuracy of 36 %, RRSLVQ improved the prediction to an accuracy of 73 %, which is approximately equal to the performance of ARF. On the artificial data streams, the RSLVQ performs worst with a mean accuracy of 69 %. The RRSLVQ does a better job on the synthetic concept drift streams due to the drift detector and thus has better accuracy than RSLVQ and HAT with 81 %. Again, SAMKNN performed best with an accuracy of 86 %. Overall, RRSLVQ performed approximately equal to OZA with an



(a) Incremental Time Mixed$_a$ (b) Incremental Time Mixed$_{ra}$

Fig. 7. Incremental time in seconds per amount of samples processed on Mixed generator with abrupt drift (left) and frequent reoccurring drift (right). All tested methods are increasing linearly in time due to linear complexity per batch, even during drift.

Table 4
Interleaved mean accuracy on data streams. Moving average of accuracy on one million samples. Winner marked bold. a = abrupt, if = incremental fast, im = incremental medium, g = gradual, ra = frequent reoccurring

Streams	ARF [15]	SAMKNN [7]	HAT [26]	RSLVQ [2]	RRSLVQ	OZA [27]
COV	0.699	**0.8937**	0.8221	0.3647	0.7278	0.6909
ELEC	**0.856**	0.7250	0.7750	0.6220	0.6448	0.740
POKER	**0.8071**	0.7951	0.6455	0.7206	0.5779	0.7904
WTHR	0.7392	0.7784	0.6771	0.6502	0.6703	**0.7827**
GMSC	**0.930**	0.9270	0.8800	0.7800	0.9213	0.9196
SQR	0.5114	**0.9659**	0.7296	0.334	0.5993	0.4439
REAL MEAN	0.7572	**0.8476**	0.7548	0.5787	0.6838	0.7556
MIXED$_a$	0.934	**0.990**	0.870	0.733	0.8930	0.975
MIXED$_g$	0.912	**0.990**	0.870	0.733	0.8938	0.975
Hyperplane	0.5091	0.5602	0.622	0.5254	**0.6208**	0.5731
RTG	0.5827	0.7008	0.6582	0.5769	**0.8075**	0.656
RBF$_{if}$	0.7457	**0.9371**	0.6242	0.5939	0.6648	0.9353
RBF$_{im}$	0.7983	0.9389	0.7344	0.5992	0.6797	**0.9557**
SEA$_a$	0.8866	0.8898	0.8362	0.8077	**0.8924**	0.881
SEA$_g$	0.8764	0.8869	0.8295	0.8033	**0.8996**	0.8747
MIXED$_{ra}$	0.8492	**0.8535**	0.5288	0.7316	0.7794	0.5464
MIXED$_{rg}$	**0.8816**	0.8535	0.5288	0.7227	0.7790	0.5468
SEA$_{ra}$	0.8668	0.8784	0.8312	0.8019	0.8857	**0.9471**
SEA$_{rg}$	0.851	0.8808	0.8273	0.8049	0.8908	**0.9471**
ARTIFICIAL MEAN	0.8078	**0.8633**	0.73	0.6899	0.8072	0.8178
OVERALL MEAN	0.7909	**0.8581**	0.7383	0.6528	0.7455	0.7971

Table 5
Interleaved test-then-train Kappa statistics on data streams. Moving average of Kappa on one million samples. Winner marked bold. a = abrupt, if = incremental fast, im = incremental medium, g = gradual, ra = frequent reoccurring

Streams	ARF [15]	SAMKNN [7]	HAT [26]	RSLVQ [2]	RRSLVQ	OZA [27]
COV	0.4267	**0.8285**	0.7139	0	0.4598	0.8233
ELEC	**0.6981**	0.4334	0.5347	0.2158	0.2616	0.4609
POKER	**0.6673**	0.6259	0.3689	0.4969	0.3498	0.6162
WTHR	0.3352	0.4439	0.2634	0.268	0.2743	**0.47**
GMSC	**0.0788**	0.002	0.0069	-0.0019	0.0017	0.0026
SQRE	0.3486	**0.9546**	0.6394	0.112	0.209	0.2054
REAL MEAN	0.4258	**0.5481**	0.4212	0.1818	0.2594	0.4297
MIXED$_a$	0.8685	**0.9791**	0.7392	0.4643	0.8117	0.9505
MIXED$_g$	0.823	**0.9791**	0.7392	0.4643	0.8138	0.9505
Hyperplane	0.0175	0.1204	0.2441	0.0508	**0.2555**	0.1462
RTG	0.1631	**0.3441**	0.273	0.1315	0.2574	0.1723
RBF$_{if}$	0.4701	**0.8736**	0.2434	0.1861	0.2457	0.8658
RBF$_{im}$	0.594	0.8773	0.4297	0.1601	0.2831	**0.9063**
SEA$_a$	**0.7454**	0.7282	0.6029	0.5434	0.7398	0.7082
SEA$_g$	0.723	0.7626	0.6435	0.5905	**0.7896**	0.7376
MIXED$_{ra}$	0.6982	**0.707**	0.0576	0.2998	0.4445	0.0928
MIXED$_{rg}$	**0.7631**	0.707	0.0576	0.2989	0.4313	0.0936
SEA$_{ra}$	0.6997	0.7143	0.609	0.5780	0.7324	**0.8851**
SEA$_{rg}$	0.6561	0.7426	0.6286	0.5802	0.7646	**0.8851**
ARTIFICIAL MEAN	0.6018	**0.7113**	0.439	0.3623	0.5474	0.6162
OVERALL MEAN	0.5431	**0.6569**	0.4331	0.3022	0.4515	0.554

accuracy of 81 %, which is a performance increase of 12 % compared to the baseline RSLVQ.

Comparing the prediction results in Table 4 to the time and memory consumption in Fig. 7 and 6, it encourages the assumption that there is a trade-off between prediction performance and model size and time.

The results of the stream experiment measured by Kappa statistics are shown in Table 5. The improved performance of RRSLVQ compared to RSLVQ is also given w.r.t. Kappa. This means that RRSLVQ also performs better on imbalanced data and has no tendency to only predict one label. Especially on COVTYPE, where RSLVQ was not able to separate the classes of the dataset, RRSLVQ showed a Kappa score of 45.98 %. The improved score is not only related to frequent reoccurring drift. It is also given when using other types of drift. On the Hyperplane generator, RRSLVQ performed best and is the only algorithm with a Kappa score > 0.2. This is due to the case that the distribution of the stream changes naturally very often and thus needs a sensible concept drift detector. However, the Kappa score is still very low on this dataset. Given the performance of the baseline classifier, the RRSLVQ does a good job of improving the RSLVQ on concept drift streams. On the real-world streams, the Kappa score is also better, but the overall improvement is small. This is affected by the fact that RSLVQ did a better job of distinguishing the ten classes of the POKER dataset. Overall, SAMKNN performed best with a score of 65 %, while RRSLVQ performed very similar to HAT at 45 %

Summarizing, the advantages of the RRSLVQ are the stability during concept drift and the constant model size and time. This makes the processing of stream data on a limited technical device feasible. The prototypes are providing an interpretable model, which is a lacking feature of the competitive algorithms. The prediction performance is comparable with OZA and HAT.

6. Conclusion

The proposed method is a major improvement to the original RSLVQ. Especially in streams with high rates of drift, the RRSLVQ[1] shows remarkable stability over time.

Kswin seems to detect occurring changes in stream data and supports the concept drift handling process with good indicators at a given time. Based on the experimental setting, the Kswin algorithm is the best at detecting drift at the given streaming data.

[1] Source code available at https://github.com/ChristophRaab/rrslvq

Further, the adaptation strategy paired with the momentum-based gradient descent is useful to minimize performance losses during the drift.

Compared to other stream approaches, the RRSLVQ provides a straightforward and interpretable model. The memory and time complexity is easy to bound and well-suited for embedding systems. The prediction performance is comparable with OzaBaggingAdwin and Hoeffding Adaptive Tree.

Future work should tackle dimension-wise testing at every time step to avoid unneeded tests. Besides, Kswin should be combined with other classifiers and false positives should be inspected. Although that Kswin already outperforms standard detectors, the task of concept drift detection must still be improved.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

We are thankful for support in the FuE program Informations- und Kommunikationstechnik of the StMWi, project OBerA, grant number IUK-1709-0011// IUK530/010.

Appendix A. Dataset Description

In this appendix, the data streams are described. It is split up in synthetic data streams and real-world streams.

A1. Synthetic Stream Generators

The synthetic stream generators are potentially infinite and drift can be implemented by changing the generating function. A common approach to create drift is to invert the function. The length of the streams is set to one millions time steps. **SEA** The SEA generator is an implementation of the data stream with abrupt concept drift, first described by Street and Kim in [29]. It produces data streams with three continuous attributes (f_1, f_2, f_3). The range of values that each attribute can assume lies between 0 and 10. Only the first two attributes (f_1, f_2) are relevant, i.e. f_3 does not influence the class value determination. New instances are obtained through randomly setting a point in a two-dimensional space, such that these dimensions correspond to f_1 and f_2. This two-dimensional space is split into four blocks, each of which corresponds to one of four different functions. In each block, a point belongs to class 1 if $f_1 + f_2 \leq \theta$ and to class 0 otherwise. The threshold θ is used to split instances between class 0 and 1 assumes the values 8 (block 1), 9 (block 2), 7 (block 3), and 9.5 (block 4). Two important features are the possibility to balance classes, which means the class distribution will tend to a uniform one, and the possibility to add noise, which will, according to some probability, change the chosen label for an instance. In this experiment, the SEA generator is used with 10 % noise in the data stream. SEA_g simulates one gradual drift, while SEA_a simulates an abrupt drift.

MIXED The MIXED Generator creates a binary classification stream. The stream consists of four features, two Boolean attributes v, w and two numeric attributes x and y between [0; 1]. The label is positive if two out of three conditions are satisfied

$$v = true, t = true, y < 0.5 + 0.3 \sin(3\pi x). \tag{A.1}$$

After each concept drift, the classification is reversed [25].

RTG The Random Tree Generator (RTG) is based on a random tree that splits features at random and sets labels to its leafs [30]. After the tree is built, new instances are obtained through the assignment of uniformly distributed random values to each attribute. The leaf reached after a traverse of the tree, according to the attribute values of an instance, determines its class value. RTG allows customizing the number of nominal and numeric attributes, as well as the number of classes. In our experiments, we did not simulate drifts for the RTG data set. Since the concepts are generated and classified according to a tree structure, in theory, it should favor decision tree learners.

RBF This generator produces data sets by means of the Radial Basis Function (RBF) [22]. This generator creates several centroids, having a random central position and associates them with a standard deviation value, a weight, and a class label. To create new instances, one centroid is selected at random, where centroids with higher weights have more chances to be selected. The new instance input values are set according to a random direction chosen to offset the centroid. The extent of the displacement is randomly drawn from a Gaussian distribution according to the standard deviation associated with the given centroid. Incremental drift is introduced by moving centroids at a continuous rate, effectively causing new instances that ought to belong to one centroid to another with (maybe) a different class. Both RBF_m and RBF_f were parametrized with 50 centroids, and all of them drift. RBF_m simulates a moderate incremental drift (speed of change set to 0.0001) while RBF_f simulates a faster incremental drift (speed of change set to 0.001).

HYPER The HYPER data set simulates an incremental drift, and it was generated based on the hyperplane generator [31]. A hyperplane is a flat, $n-1$ dimensional subset of that space that divides it into two disconnected parts. It is possible to change a hyperplane orientation and position by slightly changing its relative size of the weights w_i. This generator can be used to simulate time-changing concepts by varying the values of its weights as the stream progresses [32]. HYPER was parametrized with ten attributes and magnitude of change of 0.001. Also, a 10 % noise was added.

A2. Real-world Streams

The real-world streams are finite in length and concept drift from a statistical standpoint is unknown. However, some streams have scenarios, which change by nature like COVTYPE.

GMSC The Give Me Some Credit (GMSC) data set[2] is a credit scoring data set where the objective is to decide whether a loan should be allowed or not. This decision is essential for banks since erroneous loans lead to the risk of default and unnecessary expenses on future lawsuits. The data set contains historical data on 150,000 borrowers, each described by ten attributes.

Electricity The Electricity data set[3] was collected from the Australian New South Wales Electricity Market, where prices are not fixed. These prices are affected by the demand and supply of the market itself and set every five minutes. The Electricity data set contains 45,312 instances, where class labels identify the changes in the price (two possible classes: up or down) relative to a moving average of the last 24 hours. An important aspect of this data set is that it exhibits temporal dependencies [25].

Poker-Hand The Poker-Hand data set consists of 1,000,000 instances and eleven attributes. Each record of the Poker-Hand data set is an example of a hand consisting of five playing cards drawn from a standard deck of 52. Each card is described using two attributes (suit and rank), for a total of ten predictive attributes. There is one class attribute that describes the *Poker Hand*. This data set has no drift in its original form since the poker hand definitions do not change, and the instances are randomly generated.

[2] https://www.kaggle.com/c/GiveMeSomeCredit
[3] https://www.openml.org/d/151

Thus, the version presented in [33] is used, in which virtual drift is introduced via sorting the instances by rank and suit. Duplicate hands were removed.

Forest Cover Type Forest Cover Type (COVTYPE) is a data set, which is often used to benchmark stream mining algorithms [33,34]. The dataset assigns cartographic variables like elevation, soil type, slope, etc. of 30 square meter cells to different forest cover types. It only includes forests with minimal human-caused disturbances, so that the contained cover types are mostly a result of ecological processes.

Airlines The task on the Airlines dataset is to predict whether a planned flight will be delayed or not. It contains the airport of departure and arrival, as well as the airline and time-related features. The delay is only represented as a binary attribute. The dataset is used in the form as the MOA framework [32] provides it.

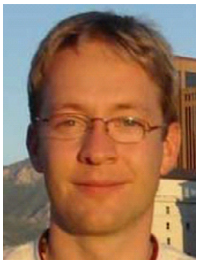
SQRE The Moving Squares dataset has been used in [7] and consists of four equally distant separated squared uniform distributions which are moving in a horizontal direction with constant speed. Whenever the leading square reaches a predefined boundary, the direction is inverted. Each square represents a different class. The added value of this dataset is the predefined time horizon of 120 examples before old instances may start to overlap current ones. Thus, the dataset should be useful for dynamic sliding window approaches, allowing testing whether the size is adjusted accordingly.

References

[1] J. Gama, I. Zliobaite, A. Bifet, M. Pechenizkiy, A. Bouchachia, A survey on concept drift adaptation, ACM Computing Surveys 46 (4) (2014) 1–37.
[2] S. Seo, M. Bode, K. Obermayer, Soft nearest prototype classification, IEEE Transactions on Neural Networks 14 (2) (2003) 390–398.
[3] M. Straat, F. Abadi, C. Göpfert, B. Hammer, M. Biehl, Statistical mechanics of on-line learning under concept drift, Entropy 20 (10) (2018) 775.
[4] A. Bifet, R. Gavaldà, Kalman Filters and Adaptive Windows for Learning in Data Streams, in: L. Todorovski, N. Lavrač, K.P. Jantke (Eds.), Discovery Science, Springer Berlin Heidelberg, Berlin, Heidelberg, 2006, pp. 29–40.
[5] D.M. dos Reis, P. Flach, S. Matwin, G. Batista, Fast Unsupervised Online Drift Detection Using Incremental Kolmogorov-Smirnov Test, Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining - KDD '16 22 (1) (2016) 1545–1554.
[6] C. Salperwyck, M. Boullé, V. Lemaire, et al., Concept drift detection using supervised bivariate grids, Proceedings of the International Joint Conference on Neural Networks 2015-September (2015).
[7] V. Losing, B. Hammer, H. Wersing, KNN classifier with self adjusting memory for heterogeneous concept drift, Proceedings - IEEE International Conference on Data Mining, ICDM 1 (2017) 291–300.
[8] J. Climer, M.J. Mendenhall, Dynamic Prototype Addition in Generalized Learning Vector Quantization, Advances in Self-Organizing Maps and Learning Vector Quantization 428 (2016) 355–368.
[9] D. Nova, P.A. Estévez, A review of learning vector quantization classifiers, Neural Computing and Applications 25 (3-4) (2014) 511–524.
[10] A. Sato, K. Yamada, Generalized Learning Vector Quantization, Advances in Neural Information Processing Systems, NIPS (8) (1995) 423–429.
[11] T. Villmann, A. Bohnsack, M. Kaden, Can learning vector quantization be an alternative to SVM and deep learning? - Recent trends and advanced variants of learning vector quantization for classification learning, Journal of Artificial Intelligence and Soft Computing Research 7 (1) (2017) 65–81.
[12] A. Zell, G. Mamier, R. Hübner, N. Schmalzl, T. Sommer, M. Vogt, SNNS: An Efficient Simulator for Neural Nets, in: MASCOTS '93, Proceedings of the International Workshop on Modeling, Analysis, and Simulation On Computer and Telecommunication Systems, January 17-20, 1993, La Jolla, San Diego, CA, USA, 1993, pp. 343–346.
[13] V. Losing, B. Hammer, H. Wersing, Interactive online learning for obstacle classification on a mobile robot, Proceedings of the International Joint Conference on Neural Networks 2015-September (2) (2015).
[14] A. Bifet, J. Read, G. Holmes, Efficient Online Evaluation of Big Data Stream Classifiers Categories and Subject Descriptors, Proceedings of the 21th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (2015) 59–68.
[15] H.M. Gomes, A. Bifet, J. Read, J.P. Barddal, F. Enembreck, B. Pfharinger, G. Holmes, T. Abdessalem, Adaptive random forests for evolving data stream classification, Machine Learning 106 (9) (2017) 1469–1495.
[16] J.a. Gama, P. Medas, G. Castillo, P. Rodrigues, Learning with Drift Detection, in: A.L.C. Bazzan, S. Labidi (Eds.), Advances in Artificial Intelligence – SBIA 2004, Springer Berlin Heidelberg, Berlin, Heidelberg, 2004, pp. 286–295.
[17] K. Wankhade, T. Hasan, R. Thool, A Survey: Approaches for Handling Evolving Data Streams, in: 2013 International Conference on Communication Systems and Network Technologies, 2013, pp. 621–625.
[18] R.H.C. Lopes, Kolmogorov-Smirnov Test, Springer Berlin Heidelberg, Berlin, Heidelberg, 2011, pp. 718–720.
[19] H. Abdi, The Bonferonni and Šidák Corrections for Multiple Comparisons, Encyclopedia of Measurement and Statistics (2007) 103–107.
[20] M. Heusinger, C. Raab, F.-M. Schleif, Passive Concept Drift Handling via Momentum Based Robust Soft Learning Vector Quantization, in: A. Vellido, K. Gibert, C. Angulo, J.D. Martín Guerrero (Eds.), Advances in Self-Organizing Maps, Learning Vector Quantization, Clustering and Data Visualization, Springer International Publishing, Cham, 2020, pp. 200–209.
[21] M. Biehl, F. Abadi, C. Göpfert, B. Hammer, Prototype-Based Classifiers in the Presence of Concept Drift: A Modelling Framework, in: Advances in Self-Organizing Maps, Learning Vector Quantization, Clustering and Data Visualization - Proceedings of the 13th International Workshop, WSOM+ 2019, Barcelona, Spain, June 26-28, 2019, 2019, pp. 210–221.
[22] J. Montiel, J. Read, A. Bifet, T. Abdessalem, Scikit-Multiflow: A Multi-output Streaming Framework, Journal of Machine Learning Research 1 (2018) 1–5.
[23] A. Cornuéjols, On-Line Learning: Where Are We So Far? in: Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics), volume 6202 LNAI, Springer, 2010, pp. 129–147.
[24] M. Baena-García, J. Del Campo-Ávila, R. Fidalgo, A. Bifet, R. Gavaldà, R. Morales-Bueno, Early Drift Detection Method, Fourth International Workshop on Knowledge Discovery from Data Streams 6 (2006) 77–86.
[25] J.a. Gama, P. Medas, G. Castillo, P. Rodrigues, Learning with Drift Detection, in: A.L.C. Bazzan, S. Labidi (Eds.), Advances in Artificial Intelligence – SBIA 2004, Springer Berlin Heidelberg, Berlin, Heidelberg, 2004, pp. 286–295.
[26] A. Bifet, R. Gavaldà, Adaptive Learning from Evolving Data Streams, in: Advances in Intelligent Data Analysis VIII, Springer Berlin Heidelberg, Berlin, Heidelberg, 2009, pp. 249–260.
[27] R.S. Oza Nikunj, N.C. Oza, Online bagging and boosting, in: Proceedings of the IEEE International Conference on Systems, Man and Cybernetics, Waikoloa, Hawaii, USA, October 10-12, 2005, volume 3, IEEE, 2005, pp. 2340–2345.
[28] V. Losing, B. Hammer, H. Wersing, Self-adjusting memory: How to deal with diverse drift types, IJCAI International Joint Conference on Artificial Intelligence (2017) 4899–4903.
[29] W.N. Street, Y. Kim, A streaming ensemble algorithm (SEA) for large-scale classification, in: Proceedings of the Seventh ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, in: KDD '01, ACM, New York, NY, USA, 2001, pp. 377–382.
[30] P.M. Domingos, G. Hulten, Mining high-speed data streams, in: Proceedings of the sixth ACM SIGKDD international conference on Knowledge discovery and data mining, Boston, MA, USA, August 20-23, 2000, pp. 71–80.
[31] G. Hulten, L. Spencer, P. Domingos, Mining time-changing data streams, in: Proceedings of the Seventh ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, in: KDD '01, ACM, New York, NY, USA, 2001, pp. 97–106.
[32] A. Bifet, R. Gavaldà, G. Holmes, B. Pfahringer, Machine Learning for Data Streams with Practical Examples in MOA, MIT Press, 2018.
[33] A. Bifet, B. Pfahringer, J. Read, G. Holmes, Efficient Data Stream Classification via Probabilistic Adaptive Windows, in: Proceedings of the 28th Annual ACM Symposium on Applied Computing, in: SAC '13, ACM, New York, NY, USA, 2013, pp. 801–806.
[34] N.C. Oza, S. Russell, Experimental comparisons of online and batch versions of bagging and boosting, Proceedings of the seventh ACM SIGKDD international conference on Knowledge discovery and data mining - KDD '01 (2001) 359–364.
[35] C. Raab, F.-M. Schleif, Reactive Soft Prototype Computing for frequent reoccurring Concept Drift, in: 27th European Symposium on Artificial Neural Networks, ESANN 2019, Bruges, Belgium, April 24-26, 2019, 2019, pp. 437–442.

Christoph Raab received his B. Eng. degree in Computer Science and his M. Sc. in Information Systems from University of Applied Sciences Wuerzburg-Schweinfurt. He is a Ph. D. student in the joint research project between the Cluster of Excellence Cognitive Interaction Technology Institute of University Bielefeld and University of Applied Science Wuerzburg-Schweinfurt since early 2018. His research interests are in machine learning, in particular learning in non-stationary environments and online and probabilistic learning.

Moritz Heusinger received his B. Sc. and M. Sc. degree from the University of Applied Sciences Wuerzburg-Schweinfurt, Germany. He is a Ph. D. student in the joint research project of the Cluster of Excellence Cognitive Interactive Technology (CITEC) Bielefeld and the University of Applied Science Wuerzburg-Schweinfurt since 2019. His main areas of research interest are learning high dimensional data and data preprocessing in non-stationary environments.



Frank-Michael Schleif(Dipl.-Inf, University of Leipzig, PhD, TU-Clausthal, Germany) was a Marie Curie Senior Research Fellow at the University of Birmingham, Birmingham, UK and a Post-Doctoral Fellow in the group of Theoretical Computer Science (TCS) at the University of Bielefeld, Bielefeld, Germany, whxere he also received a venia legendi in applied computer science in 2013. He was also a software developer and consultant for the Bruker Corp. Since 2016 he is with the University of Applied Sciences, Wuerzburg, Germany, where he is a Professor for Database Management and Business Intelligence. His current research interests include data management, computational intelligence techniques and machine learning for non-metric models and large scale problems. Several research stays have taken him to UK, the Netherlands, Japan and the USA. He is a member of the German chapter of the European Neural Network Society (GNNS), the GI and the IEEE-CIS. He is editor of the Machine Learning Reports and member of the editorial board of the Neural Processing Letters.